

Secure System Development Cycle

How security was built into Kasi Grid, phase by phase

Amethyst & Azure

Kasi Grid: Know before it goes dark.

FNB App of the Year Hackathon 2026 · Track: Climate resilience and essential services

Tshedza Tshikovhi and Mulanga Munaka

Source code: https://github.com/Unsolvedstar/Kasi-Grid

Live app: https://kasi-grid.vercel.app

Demo video: https://drive.google.com/file/d/1vZBvpxMXeu8RfhizZllfzZQadJr1p1d-/view?usp=drive_link

1. Security requirements (risk assessment)

Principle: collect as little personal data as possible and never trust one report. Anyone can look around without an account, and to take part a person signs in and uses a name their neighbours can see. The account stores only that name, a cell number or email (never shown to others), a salted password hash and a few settings. Deleting the account removes the name from every report and post. Location is optional and only ever shared when the person taps a button.

Risk	Likelihood / impact	Control
Fake or spam reports	High / High	An outage is verified only when 3 different devices agree. Write endpoints are rate limited and text length is capped.
Abuse and denial of service	Medium / Medium	Global API limit and a stricter limit on writes. The app works offline, so people still see their last data.
Unauthorised admin access	Medium / High	The admin area is off unless a secret token is set; it is compared in constant time and rate limited.
Privacy (location, identity, health)	Medium / High	No names are needed. Reports are tied to a community; an exact pin is optional and board posts are shown only to about 100 m. Medical battery details stay on the phone. A POPIA review is planned before launch.
Leaked secrets	Medium / High	Keys and tokens live in server environment variables only.
Unsafe medical guidance	Medium / High	Battery estimates are cautious and show their assumptions; the app prompts a call to the ambulance and says it is not medical advice.
Malicious carer or supplier	Low / High	Vetting through a partner and admin approval before listing; quick removal.
Account takeover and password guessing	Medium / High	Passwords are stored only as salted scrypt hashes and never returned. Sign-in is rate limited, and an unknown account takes as long to refuse as a wrong password. Sessions are random tokens in an HttpOnly, SameSite cookie for 90 days; only a hash of the token is stored. A person can sign out or delete the account (and every session) in the app. The medical plan and emergency contact never leave the phone.
Injection and cross-site scripting	Low / High	Database queries use parameters; React renders user text as text.

Residual risk: coordinated fake reports from many devices. Accepted for the pilot, watched with the admin moderation tools, and reduced by raising the verification threshold in sensitive areas.

2. Threat modelling and design review

Method: STRIDE across the phone app, the API, the database, the admin page and the third-party data sources.

Threat	Where it applies and what we do
Spoofing	One account can confirm an outage only once, so a fake outage needs three real accounts. Accounts use salted password hashes, rate-limited sign-in and HttpOnly session cookies. The admin page uses a secret token.
Tampering	All database queries are parameterised. Reports are validated and trimmed on the server. Prices for bookings are set on the server, never by the client.
Repudiation	Moderation actions are limited to the admin page.
Information disclosure	Names are public to the community by design, but numbers and emails are not. Optional pins, board posts rounded to about 100 m, secrets never sent to the browser. Bookings are private to a device and never cached by the service worker.
Denial of service	Rate limits, request size limits, an offline-capable app.
Elevation of privilege	The admin area answers "not found" unless it is switched on.

Design decisions that reduce risk: the browser never calls the data providers directly for protected data (the server does); verification needs three devices; the medical plan never leaves the phone.

3. Development (secure coding practices)

- Secrets in environment variables, never in code or the browser; `.env` is git-ignored.
- Parameterised SQL everywhere; input validated on the server (length, type allow-lists, no links, no phone or ID numbers in public posts).
- React renders user text as text, which blocks script injection.
- Timing-safe comparison for the admin token; security headers (nosniff, frame deny, referrer policy).
- Per-IP rate limits on the API, writes and admin routes; request bodies capped at 10 KB.
- Small, separated modules for shared logic, each with unit tests; every change is a reviewed commit in git.
- Accessibility built in from the start (screen-reader text, large text, flash and vibration alerts), because a safety alert nobody can perceive is a safety failure.

4. Security testing

- **112 automated tests passing** (`npm test`): calculations, translations, the full API including admin, care bookings, rate limits and the outage rules, community and location rules, the battery model and alert stages, and accounts (registration, sessions, sign-out, settings sync, bookings following a person, deletion).
- **Dependency scan (`npm audit`):** run during the build of this submission. All findings in production dependencies (body-parser, ip-address and the PDF library) were fixed by updating, and the production scan now reports **0 vulnerabilities**. Two moderate/high findings remain in the development-only build tool (Vite), which is not shipped to users.
- **Manual and browser testing:** every main flow was run in a real browser: wizards, report and booking submission, location and pin rules (pins outside South Africa or far from the community are rejected), the battery countdown and ambulance banner, and the emergency card.

- **Planned before launch:** an independent penetration test, a Content-Security-Policy header, load testing of the write endpoints, and a backup and restore test.

5. Assessment and secure integration

Overall risk for a pilot: low to medium. The app holds no passwords, payment details or precise home addresses, so a breach exposes little.

- **Data providers** (EskomSePush, weather, OpenStreetMap) are called by the server with keys kept server-side, and answers are cached to save quota. Building outlines for the map come through our own server so the public service is not hit by every visitor.
- **Hosting:** the front end is static and the API runs as a serverless function; data lives in a hosted libSQL database reached with its own token over an encrypted connection. Serverless means no long-lived server to patch, and rate limits are per instance (a known limit, noted in the deployment guide).
- **Payments:** none are taken yet. Carers, equipment prices and bookings are labelled samples. Before real payments we will use a certified provider so card details never touch our servers.
- **Open items before real users:** a POPIA review, a Content-Security-Policy header, a backup and restore test and an independent security check.